

Bancos de Dados

Aula - 04.1

2026

SENAI - CRTI

Operações de Escrita (Create) no MongoDB

Unidade Curricular: **Bancos de Dados**



Agenda da Aula

01

Situação de Aprendizagem

Contextualização do problema real

02

Mobilização de Conhecimentos

Ativação do que já foi aprendido

03

Desenvolvimento Teórico

Conteúdo aplicado com exemplos

04

Práticas Progressivas

7 exercícios com complexidade crescente

05

Sistematização + Avaliação

Resumo visual e atividade final individual

O Problema do Desenvolvedor

O Cenário

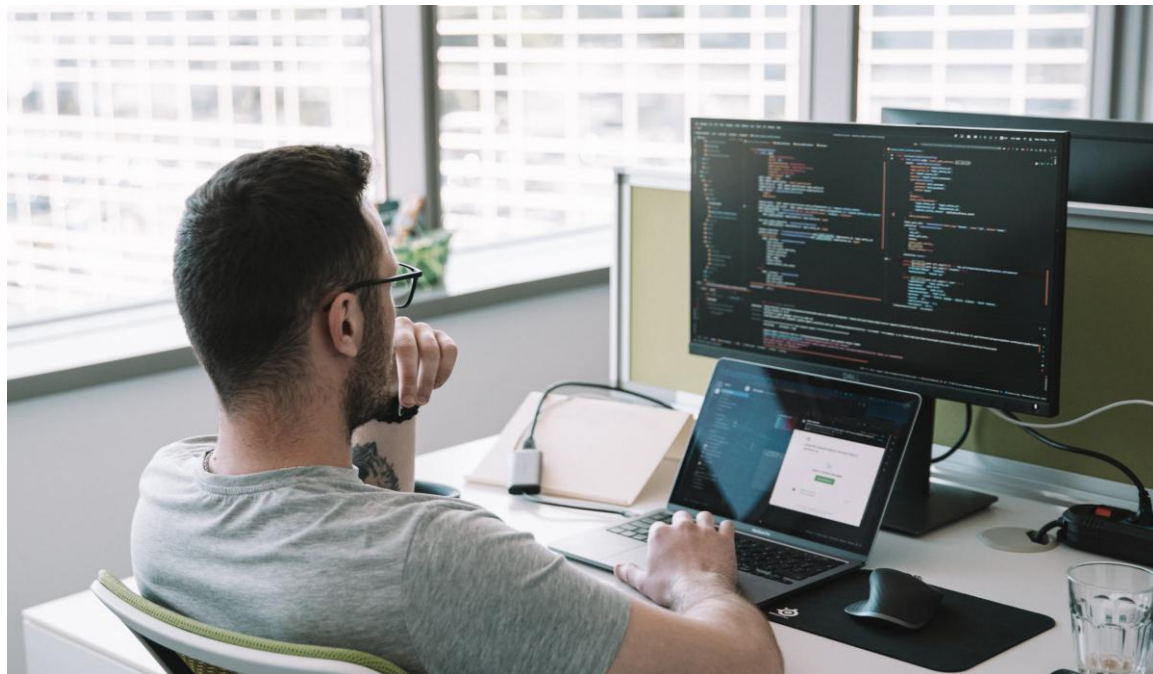
Uma startup está construindo um sistema de e-commerce com **Node.js e MongoDB**. O banco de dados já foi instalado e a conexão está funcionando — mas nenhum dado foi inserido ainda.

O desenvolvedor júnior precisa armazenar **usuários, produtos e pedidos**, mas não sabe como realizar operações de escrita no MongoDB.

O Problema Central

- ✗ Sem saber inserir dados, a aplicação não funciona.
- ✗ O banco está vazio — nenhuma consulta retorna resultados.
- ✗ Os dados chegam em formato JSON do front-end, mas não há domínio do comando correto para persistência.

 Ao final desta aula, você será capaz de resolver exatamente esse problema!



MOBILIZAÇÃO DE CONHECIMENTOS PRÉVIOS

O Que Você Já Sabe?

Antes de avançar, vamos ativar o que você já conhece. Reflita honestamente sobre cada pergunta:

Perguntas de Ativação

 Como adicionar dados em um banco?

Em SQL usamos `INSERT INTO tabela VALUES (...);`. No MongoDB, a lógica é parecida, mas a sintaxe é diferente e os dados são armazenados como **documentos**, não linhas de tabela.

 Qual a estrutura de um documento?

Um documento MongoDB é baseado em **JSON** (JavaScript Object Notation): pares de chave-valor entre chaves `{ }`. Ex: `{ "nome": "João", "idade": 25 }`. Internamente, o MongoDB usa **BSON** (Binary JSON).

 Você já utilizou JSON?

JSON está em todo lugar: respostas de APIs REST, arquivos de configuração (`package.json`), comunicação entre front-end e back-end. Se você já trabalhou com Node.js, já usou JSON!

JSON e BSON: A Base dos Documentos

Estrutura JSON

Todo documento MongoDB é escrito em formato JSON. Cada campo possui um nome (chave) e um valor, separados por dois-pontos.

```
{
  "nome": "João Silva",
  "idade": 25,
  "ativo": true,
  "email": "joao@email.com"
}
```

Tipos de valores suportados: **string**, **número**, **booleano**, **array**, **objeto aninhado** e **null**.

Por que BSON?

O MongoDB armazena os documentos internamente em **BSON (Binary JSON)**, um formato binário que:

- Suporta mais tipos de dados que o JSON puro
- É mais eficiente para leitura e escrita
- Adiciona tipos como `ObjectId`, `Date` e `BinData`

 O campo `_id` (`ObjectId`) é gerado automaticamente pelo MongoDB em toda inserção.

O Comando `insertOne()`

O `insertOne()` é utilizado para inserir **um único documento** em uma coleção. Se a coleção não existir, o MongoDB a cria automaticamente.

Sintaxe

```
db.colecao.insertOne({ campo: valor })
```

Exemplo Prático

```
db.usuarios.insertOne({  
  nome: "João",  
  idade: 25,  
  email: "joao@email.com"  
});
```

Retorno Esperado

```
{  
  acknowledged: true,  
  insertedId: ObjectId(  
    "64a1f3e2b5c9d..."  
  )  
}
```

Pontos de Atenção

- `acknowledged: true` confirma que o documento foi inserido
- `insertedId` retorna o `_id` gerado
- Você pode definir seu próprio `_id` se necessário

O Comando `insertMany()`

O `insertMany()` permite inserir **múltiplos documentos de uma só vez**, passando um array. É muito mais eficiente do que chamar `insertOne()` repetidamente.

Sintaxe

```
db.colecao.insertMany([
  { campo: valor },
  { campo: valor }
])
```

Exemplo Prático

```
db.usuarios.insertMany([
  { nome: "Ana",    idade: 20 },
  { nome: "Carlos", idade: 30 }
]);
```

Retorno Esperado

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("64a1..."),
    '1': ObjectId("64a2...")
  }
}
```

 Por padrão, o `insertMany()` insere documentos em ordem. Use `{ ordered: false }` para continuar mesmo se um documento falhar.

insertOne vs insertMany

insertOne()

- Insere **1 documento**
- Retorna um único `insertedId`
- Ideal para cadastros pontuais
- Ex: cadastro de um novo usuário

1

Documento por chamada

insertOne()

insertMany()

- Insere **N documentos** de uma vez
- Retorna um objeto `insertedIds`
- Ideal para carga em lote (batch)
- Ex: importar catálogo de produtos

N

Documentos por chamada

insertMany()

~3x

Mais eficiente

insertMany() em lote vs múltiplos insertOne()

Usando o mongosh: Terminal do MongoDB

O **mongosh** é o shell interativo oficial do MongoDB. É pela linha de comando que você executa todos os comandos de inserção diretamente.



Comandos Essenciais no mongosh

1

Conectar e Selecionar Banco

```
mongosh  
show dbs
```

Visualizar os bancos

2

Listar Coleções

```
use nomeDoBanco  
show collections
```

Acessar o banco e visualizar
coleções existentes.

3

Inserir e Verificar

```
db.usuarios.insertOne({...})  
db.usuarios.find()
```

Insira e confirme os dados inseridos.

Inserindo Dados em Lote no mongosh



MongoDB Command Line Database Tools

Faça o download em www.mongodb.com/try/download/database-tools. Instale.



Arquivo .json

Usando o bloco de notas, crie um arquivo com estrutura de array .json e salve na pasta C:\Program Files\MongoDB\Tools\100\bin



Navegue ou insira no path o mongoimport

Insira o arquivo mongoimport presente em C:\Program Files\MongoDB\Tools\100\bin ou navegue até esta pasta



Importe!

No prompt de comando, insira `mongoimport --db nomedobanco --collection nomedacoleção --file arquivo.json --jsonArray`

MongoDB Compass: Interface Gráfica

O **MongoDB Compass** é a ferramenta visual oficial do MongoDB. Permite inserir, visualizar e editar documentos sem precisar digitar comandos — ideal para quem está começando ou prefere uma interface amigável.



Inserindo Dados no MongoDB Compass



Conectar ao Servidor

Abra o Compass e conecte-se usando `mongodb://localhost:27017`. Clique em **"Connect"**.



Selecionar ou Criar Banco

Clique em **"Create Database"** e defina o nome do banco e da primeira coleção.



Inserir Documento

Dentro da coleção, clique em **"Add Data"** → **"Insert Document"** e cole ou edite o JSON.



Confirmar Inserção

Clique em **"Insert"**. O documento aparece imediatamente na lista da coleção.

Exercícios: Do Simples ao Complexo

Os exercícios seguem uma lógica de **complexidade crescente**, garantindo que cada novo desafio consolide o anterior. Você terá cerca de **90 minutos** para concluir todos.

Exercícios 1–2

Inserção básica com `insertOne()` e `insertMany()`

Exercícios 3–4

Documentos com objetos aninhados e arrays

Exercício 5

Inserção via MongoDB Compass (interface gráfica)

Exercícios 6–7

Coleções "produtos" e "pedidos" com estrutura complexa

Exercício 1

Inserindo um Documento Simples

Abra o **mongosh** e execute os comandos abaixo para criar seu primeiro banco e inserir um documento na coleção **usuarios**.

```
// Criar e selecionar o banco de dados
use minhaLoja

// Inserir um único documento
db.usuarios.insertOne({
  nome: "João Silva",
  idade: 25,
  email: "joao@email.com"
})

// Verificar a inserção
db.usuarios.find()
```

✔ ☒ Verifique se o retorno mostra **acknowledged: true** e um **insertedId**. Use **.find().pretty()** para visualizar o documento criado.

Inserindo Múltiplos Documentos

Utilize o `insertMany()` para inserir vários usuários de uma só vez. Observe como o retorno muda em relação ao `insertOne()`.

```
db.usuarios.insertMany([
  {
    nome: "Ana Costa",
    idade: 20,
    email: "ana@email.com"
  },
  {
    nome: "Carlos Mendes",
    idade: 30,
    email: "carlos@email.com"
  },
  {
    nome: "Beatriz Lima",
    idade: 28,
    email: "beatriz@email.com"
  }
])

// Contar os documentos inseridos
db.usuarios.countDocuments()
```



💡 Compare o resultado com o Exercício 1: agora o retorno inclui `insertedIds` — um array com todos os IDs gerados.

Documentos com Objetos Aninhados

O MongoDB permite que um campo contenha outro documento inteiro — chamamos isso de **documento aninhado** (ou subdocumento). Isso é muito útil para dados estruturados como endereços.

```
db.usuarios.insertOne({
  nome: "Maria Oliveira",
  idade: 32,
  endereco: {
    rua: "Av. Brasil, 1500",
    cidade: "Rio de Janeiro",
    estado: "RJ",
    cep: "20040-020"
  },
  contato: {
    telefone: "(21) 99999-0001",
    email: "maria@email.com"
  }
});
```

  Para buscar por campo aninhado, usamos a **notação de ponto**: `db.usuarios.find({ "endereco.cidade": "Rio de Janeiro" })`

Documentos com Arrays

Arrays são usados quando um campo pode ter **múltiplos valores**. No MongoDB, arrays são cidadãos de primeira classe — você pode armazenar listas de strings, números ou até outros documentos.

```
db.usuarios.insertOne({
  nome: "Pedro Souza",
  idade: 22,
  interesses: [
    "tecnologia",
    "esportes",
    "música"
  ],
  habilidades: [
    { linguagem: "JavaScript", nivel: "avançado" },
    { linguagem: "Python", nivel: "intermediário" }
  ]
});

// Buscar por item no array
db.usuarios.find({
  interesses: "tecnologia"
});
```

● Exercício 5

Inserção via MongoDB Compass

Agora vamos repetir a inserção de um usuário, mas desta vez utilizando a **interface gráfica do MongoDB Compass**. Compare a experiência com o terminal.

1 Abrir o Compass e conectar em `localhost:27017`

Clique no banco `minhaLoja` que você criou nos exercícios anteriores.

2 Acessar a coleção `usuarios`

Clique na coleção e depois em **"Add Data" → "Insert Document"**.

3 Colar o JSON e inserir

Cole: `{ "nome": "Fernanda", "idade": 27, "cidade": "São Paulo" }` e clique em **"Insert"**.

4 Verificar o resultado visualmente

Observe o documento aparecer na lista. Note o `_id` gerado automaticamente pelo MongoDB.

Coleção "produtos"

Crie uma nova coleção chamada `produtos` e popule-a com múltiplos itens de um catálogo de loja. Pratique o uso de tipos variados de dados.

```
db.produtos.insertMany([
  {
    nome: "Notebook Dell",
    categoria: "Eletrônicos",
    preco: 3599.90,
    estoque: 15,
    disponivel: true
  },
  {
    nome: "Mouse Sem Fio",
    categoria: "Periféricos",
    preco: 89.90,
    estoque: 50,
    disponivel: true
  },
  {
    nome: "Teclado Mecânico",
    categoria: "Periféricos",
    preco: 299.00,
    estoque: 0,
    disponivel: false
  }
])
```

```
db.produtos.countDocuments()
```



Observe o uso de tipos distintos: `string`, `number` (decimal) e `boolean`.

Coleção "pedidos" — Estrutura Complexa

A coleção `pedidos` representa uma estrutura mais próxima do mundo real: um pedido contém dados do cliente, um **array de itens** e informações de entrega aninhadas.

```
db.pedidos.insertOne({
  numeroPedido: "PED-001",
  data: new Date("2024-06-10"),
  cliente: {
    nome: "João Silva",
    email: "joao@email.com"
  },
  itens: [
    { produto: "Notebook Dell", qty: 1, preco: 3599.90 },
    { produto: "Mouse Sem Fio", qty: 2, preco: 89.90 }
  ],
  total: 3779.70,
  entrega: {
    endereco: "Av. Paulista, 1000",
    cidade: "São Paulo",
    estado: "SP",
    status: "pendente"
  }
})
```

Resumo Visual: O Que Aprendemos



insertOne()

Insere **um documento** em uma coleção. Retorna `insertedId`.
Ideal para cadastros individuais em tempo real.



insertMany()

Insere **vários documentos** em lote. Retorna `insertedIds`. Mais eficiente para carga de dados em massa.



Estrutura JSON/BSON

Documentos usam pares `chave: valor`. Suportam strings, números, booleanos, arrays e objetos aninhados.

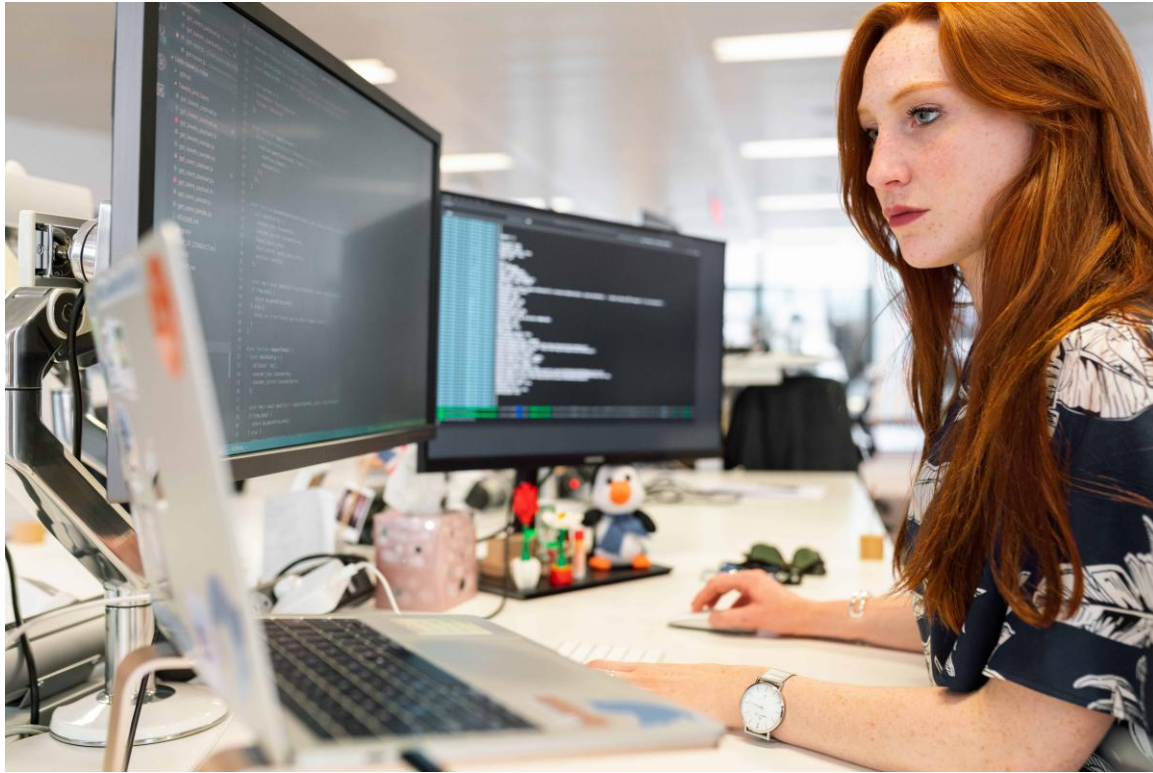


MongoDB Compass

Interface gráfica para inserção, visualização e edição de documentos — sem precisar digitar comandos.

Mapa de Comandos

Comando	Uso	Retorno
<code>use nomeBanco</code>	Seleciona ou cria um banco de dados	Mensagem de confirmação
<code>db.col.insertOne({})</code>	Insere um único documento	<code>insertedId</code>
<code>db.col.insertMany([])</code>	Insere múltiplos documentos	<code>insertedIds</code> (objeto)
<code>db.col.find()</code>	Lista todos os documentos	Cursor com documentos
<code>db.col.find().pretty()</code>	Lista documentos formatados	JSON formatado
<code>db.col.countDocuments()</code>	Conta documentos na coleção	Número inteiro
<code>show collections</code>	Lista coleções do banco atual	Nomes das coleções



ATIVIDADE FINAL — 30 MINUTOS

Criando e Populando um Banco de Dados

Situação: Você foi contratado como desenvolvedor júnior de uma loja virtual. Sua primeira tarefa é criar o banco de dados e inserir os dados iniciais.

Tarefas da Atividade Final

1

Criar o banco

Use `use loja` para criar e selecionar o banco de dados da sua loja.

2

Coleção "produtos"

Insira pelo menos **15 produtos** com campos: `nome`, `categoria`, `preco`, `estoque`. Use `insertMany()`.

3

Coleção "usuarios"

Insira pelo menos **10 usuários** com `insertOne()` e `insertMany()`.
Inclua campos aninhados (`endereço`).

4

Usar ambos os comandos

Demonstre o uso de `insertOne()` em pelo menos 1 documento e `insertMany()` nos demais.

  **Entregável:** banco `loja` criado, coleções `produtos` e `usuarios` populadas, documentos estruturados corretamente. Exporte via Compass ou mostre no mongosh com `.find()`.

